RelayGuard

Complete Project Development Guide

Predictive Maintenance System for Industrial Relays

IoT Hardware · ESP32 Firmware · Raspberry Pi Gateway · ML Pipeline · Dashboard

RelayGuard Engineering Team

Version 3.0 | April 2026

About This Document

This is the **single authoritative development guide** for the RelayGuard project. It supersedes all previous documents (*Relay_Health_System_Dev_Guide.pdf*, *RUL_Predictive_Maintenance_Solution.pdf*, *ML_Model.pdf*).
The guide covers every layer of the system – from soldering the first sensor to deploying the trained ML model – in build order, with verification checkpoints, exact pin assignments, code snippets, MODBUS register maps, MQTT topic design, and the complete updated Δ HI machine-learning architecture.
Primary ML change from v1/v2: the model now predicts *change in Health Index* (Δ HI) instead of absolute future HI, eliminating covariate shift and producing accurate predictions across the full relay lifecycle.

Contents

1. Project Vision and Architecture

1.1 What RelayGuard Does

RelayGuard is an IoT predictive maintenance system that monitors the health of industrial relays in real time and predicts their Remaining Useful Life (RUL) before failure occurs. The system achieves this by:

1. Measuring five physical signals from each relay (current, voltage, temperature, contact resistance, switching cycles).
2. Converting those signals into a single **Health Index (HI)** using physics-based degradation models running on the ESP32.
3. Transmitting HI and raw readings to a Raspberry Pi gateway over MODBUS/RS485.
4. Forwarding data to the cloud over MQTT, where an XGBoost ML model predicts future HI.
5. Displaying fleet health on a live React dashboard and auto-generating maintenance work orders.

1.2 Five-Layer Architecture

Layer	Hardware / Stack	Responsibility
L1 – Sensing	ACS712, ZMPT101B, MLX90614, INA219	Read current, voltage, temperature, contact resistance
L2 – Edge Node	ESP32-S3, FreeRTOS	Compute HI, drive LEDs/buzzer, expose MODBUS slave
L3 – Gateway	Raspberry Pi 4	Poll MODBUS, store SQLite, run ML inference, MQTT broker
L4 – Cloud	MQTT broker, InfluxDB / SQLite	Receive data, run fleet-wide XGBoost model, serve API
L5 – Dashboard	React + Recharts (or Grafana)	Fleet overview, HI timeline, work orders, RUL charts

Table 1: Five-layer system architecture.

1.3 End-to-End Data Flow

$$\underbrace{\text{Sensors}}_{\text{L1}} \xrightarrow{\text{ADC/I2C}} \underbrace{\text{ESP32}}_{\text{L2}} \xrightarrow{\text{MODBUS}} \underbrace{\text{Rasp. Pi}}_{\text{L3}} \xrightarrow{\text{MQTT/TLS}} \underbrace{\text{Cloud+ML}}_{\text{L4}} \xrightarrow{\text{HTTPS/WS}} \underbrace{\text{Dashboard}}_{\text{L5}}$$

OTA updates flow in the reverse direction: the cloud pushes refined degradation thresholds back to the Pi, which relays them to each ESP32 node.

1.4 Build Sequence at a Glance

Day	Goal	Go/No-Go Gate
0	Environment setup	Blink sketch uploads. Python + pip libraries import. Git commit pushed.
1	Sensor wiring	Each sensor reads within 5% of reference instrument.
2	ESP32 firmware	OLED shows live HI. LEDs change as test inputs change. NVS survives reboot.
3	MODBUS	QModMaster reads all 14 registers. Values match OLED.
4	Pi gateway	SQLite grows every 5 s. All node values present and sane.
5	MQTT + API	mosquitto_sub shows JSON every 5 s. /api/fleet returns JSON in browser.
6	Dashboard	HI gauge updates without page refresh. Alarm state visible within 10 s.
7	ML + integration	ml_inference.py runs on Pi. Anomaly flag in API. Full end-to-end test.

Table 2: Day-by-day build sequence with mandatory verification gates.

2. Stage 0 – Environment Setup (Day 0)

Note: Golden rule: **build layer by layer and verify each layer independently**. A sensor bug and a MODBUS bug look identical when they are stacked together. Never proceed past a verification gate until it passes.

2.1 Required Tools

Tool	Purpose	Install
Arduino IDE 2.x	ESP32 firmware	arduino.cc + Espressif ESP32 board package
Python 3.10+	Pi scripts, ML, backend	Pre-installed on Raspberry Pi OS
pymodbus	MODBUS master on Pi	`pip install pymodbus`
paho-mqtt	MQTT client on Pi	`pip install paho-mqtt`
FastAPI + uvicorn	REST API + WebSocket	`pip install fastapi uvicorn`
xgboost, scikit-learn	ML training/inference	`pip install xgboost scikit-learn joblib`
Mosquitto	MQTT broker on Pi	`sudo apt install mosquitto`
QModMaster	MODBUS test tool (PC)	sourceforge.net/projects/qmodmaster
VS Code	IDE for Python + React	code.visualstudio.com
Git	Version control	`sudo apt install git`

Table 3: Development environment tools.

2.2 ESP32 Pin Assignment

Warning: Decide your full pin map **before** soldering. Changing pins after wires are attached wastes hours. Get team agreement on this table and treat it as read-only once wiring begins.

ESP32 Pin	Module	Dir.	Purpose
GPIO 34 (ADC1)	ACS712 OUT	Input	Switched current measurement (AC/DC)
GPIO 35 (ADC1)	ZMPT101B OUT	Input	Switched voltage measurement (AC)
GPIO 21 (SDA)	INA219 SDA	Bidirectional	Contact resistance – I2C data
GPIO 22 (SCL)	INA219 SCL	Output	Contact resistance – I2C clock
GPIO 4	MLX90614 SDA	Bidirectional	Non-contact temperature sensor
GPIO 16 (RX2)	MAX485 RO	Input	MODBUS RTU receive
GPIO 17 (TX2)	MAX485 DI	Output	MODBUS RTU transmit
GPIO 18	MAX485 DE/RE	Output	RS485 direction control (HIGH = TX)
GPIO 2	Green LED	Output	Healthy state
GPIO 4	Yellow LED	Output	Warning state
GPIO 5	Red LED	Output	Critical / Imminent state
GPIO 15	Buzzer	Output	Audio alarm (PWM)
GPIO 21/22	SSD1306 OLED	Output	Local display – HI, cycles, alarm
GPIO 36 (ADC)	Relay coil GPIO	Input	Relay state detection (switching events)

Table 4: ESP32 pin assignment – finalise before wiring.

Verification Gate (must pass before next stage): Upload a blink sketch to the ESP32. Confirm Python imports on the Pi with `python -c "import pymodbus, paho.mqtt.client, fastapi"`. Push first Git commit.

3. Stage 1 – IoT Hardware Layer (Day 1)

Wire and verify each sensor *individually*. Do not connect multiple sensors until each one independently produces stable, repeatable readings verified against a reference instrument.

3.1 Current Sensor – ACS712

Wiring: Inline with the relay load circuit. ACS712 needs 5 V VCC; output must be divided to 3.3 V before the ESP32 ADC. Use a 3.3 kΩ / 6.8 kΩ voltage divider.

Verification: Zero load → ADC reads ≈2048 (12-bit mid-scale). Known resistive load → reading shifts proportionally. Average 1000 ADC samples per reading to reduce noise.

Warning: ADC input must never exceed 3.3 V. A direct 5 V output from ACS712 will damage the ESP32. Always fit the voltage divider.

3.2 Voltage Sensor – ZMPT101B

Wiring: Connected in parallel with the load (across relay output terminals). Adjust the on-board potentiometer until output is centred at ≈1.65 V with no load.

Verification: Sample at ≥1 kHz and compute RMS: $V_{\text{rms}} = \sqrt{\overline{v^2}}$. Compare to multimeter – should match within 5%.

> **Warning:** ZMPT101B outputs a bipolar AC signal centred at 1.65 V. Negative excursions below 0 V are clipped by the ADC – adjust the offset pot until the waveform is fully within 0–3.3 V.

3.3 Temperature Sensor – MLX90614

Wiring: I2C at address 0x5A. 4.7 kΩ pull-ups on SDA and SCL to 3.3 V. Aim at relay coil or contact area (non-contact IR measurement).

Verification: Point at hand vs cold surface – object temperature changes while ambient stays stable. Use Adafruit MLX90614 library.

3.4 Contact Resistance – INA219 (Most Critical)

Wiring: 4-wire (Kelvin) measurement. Inject 10 mA through contacts via a 330 Ω resistor from 3.3 V. Connect INA219 sense pins directly to contact faces. $R = V_{\text{sense}}/I_{\text{inject}}$.

Verification: New relay: 1–5 mΩ. Any reading above 50 mΩ on a new relay indicates sense wire resistance – move clips closer to contact faces.

> **Warning:** The sense wires must carry zero current. Using a single pair of wires for both current injection and voltage sense adds wire resistance to the measurement. This is the most common contact-resistance mistake.

3.5 Relay State Detection

Wiring: 10 kΩ pull-down on GPIO. Signal transistor or optocoupler driven by relay coil drive signal. GPIO HIGH = relay energised.

Verification: Toggle relay manually with a button. GPIO state follows. Implement 5 ms software debounce to avoid counting one physical switch as multiple events.

> **Warning:** Without debounce, a single relay closure generates 5–20 false edge detections due to contact bounce, inflating `N_cycles` significantly within hours.

3.6 Sensor Summary

Sensor	Module	Interface	Range	Verify Against
Current	ACS712-30A	ADC (GPIO34)	0–30 A	Clamp meter
Voltage	ZMPT101B	ADC (GPIO35)	0–250 V AC	Multimeter
Temperature	MLX90614	I2C (0x5A)	–40 to 125 °C	Thermometer
Contact-R	INA219	I2C (0x40)	0–200 mΩ	Milliohm meter
Relay State	GPIO + opto	Digital (GPIO36)	HIGH/LOW	Manual toggle

Table 5: Sensor summary with verification method.

Verification Gate (must pass before next stage): ACS712 reads known current ±5%. ZMPT101B RMS matches multimeter. MLX90614 responds to hand proximity. INA219 reads <10 mΩ for a new relay with correct 4-wire wiring. Relay state GPIO toggles cleanly with no bounce.

4. Stage 2 – ESP32 Firmware (FreeRTOS) (Day 2)

Warning: Do **not** write a single monolithic `loop()`. It will block on I2C calls and give inaccurate timing for cycle counting. Split work across two FreeRTOS tasks pinned to separate cores.

4.1 Dual-Core Task Split

	Core 0 – Sensor Task (priority 5)	**Core 1 – Health Task (priority 4)**
Runs every	100 ms	Triggered by queue item
Does	Read ACS712, ZMPT101B (1000-sample RMS), MLX90614 temp, INA219 resistance, relay GPIO edge with debounce	Compute HI sub-indices and composite HI; run alarm state machine; write MODBUS registers; update OLED (500 ms)
Data link	Sends `SensorData` struct via xQueueSend (non-blocking)	Consumes struct via xQueueReceive
NVS writes	Never	Every 100 cycles

Table 6: FreeRTOS dual-core task assignment.

Note: **Task design rules:** (1) Core 0 must never be blocked by I2C, MODBUS, or display. (2) The queue is the only legal shared data path between cores. (3) Any variable read by both cores needs a `xSemaphoreCreateMutex()`. (4) NVS flash has limited write endurance – write every 100 cycles, not every cycle.

4.2 Firmware Skeleton (Pseudo-code)

Listing 1: FreeRTOS firmware structure.

```
// Core 0 -- Sensor acquisition, priority 5
void sensor_task(void *param) {
  SensorData d;
  while(1) {
    d.I_rms    = read_acs712_rms(1000);
    d.V_rms    = read_zmpt101b_rms(1000);
    d.T_coil   = mlx.readObjectTempC();
    d.R_cont   = ina219.read_milliohms();
    d.sw_event = detect_relay_edge();   // 5 ms debounce
    xQueueSend(sensor_q, &d, 0);        // non-blocking
    vTaskDelay(pdMS_TO_TICKS(100));
  }
}
```

```
14
15 // Core 1 -- Health engine, priority 4
16 void health_task(void *param) {
17   SensorData d;
18   while(1) {
19     xQueueReceive(sensor_q, &d, portMAX_DELAY);
20     if (d.sw_event) {
21       float E  = d.I_rms * d.I_rms * d.V_rms * T_CLOSE;
22       float Kt = arrhenius_factor(d.T_coil);
23       W_total += E * Kt;
24       N_cycles++;
25       if (N_cycles % 100 == 0) nvs_save(N_cycles, W_total);
26     }
27     float HI = compute_composite_HI(W_total, d.T_coil,
28                                      d.R_cont, N_cycles);
29     update_alarm_state(HI);
30     write_modbus_registers(HI, N_cycles, d);
31     update_oled_if_due(HI, N_cycles);   // 500 ms throttle
32   }
33 }
```

Verification Gate (must pass before next stage): OLED shows live HI value. LEDs change colour as you manually change sensor inputs. NVS survives a power cycle – N_cycles and W_total are restored on reboot.

5. Stage 3 – Health Index Computation (Day 2)

The Health Index (HI) is a single number from 0% (failed) to 100% (new). It is a weighted composite of four sub-indices, each capturing a different failure mechanism.

5.1 Arc Damage Sub-Index

Arc energy per switching event:

$$E_{\text{step}} = I^2 \cdot V \cdot t_{\text{close}}$$

Accumulated with Arrhenius temperature weighting:

$$W_{\text{total}} = \sum_{\text{events}} (E_{\text{step}} \cdot K_T) \qquad K_T = \exp\left[\frac{E_a}{k_B}\left(\frac{1}{T_{\text{ref}}} - \frac{1}{T}\right)\right]$$

$$HI_{\text{damage}} = 1 - \frac{W_{\text{total}}}{W_{\text{rated}}} \quad \in [0,1] \qquad \textbf{Weight: 40\%}$$

$E_a = 0.7\,\text{eV}$, $k_B = 8.617 \times 10^{-5}\,\text{eV/K}$, $T_{\text{ref}} = 298.15\,\text{K}$.

Note: A relay switching at 85°C accumulates damage 4× faster than at 25°C. This is why temperature matters even for contact wear – not just coil insulation.

5.2 Thermal Sub-Index

$$HI_{\text{thermal}} = 1 - \frac{T_{\text{coil}}}{T_{\text{max}}} \quad \in [0,1] \qquad \textbf{Weight: 25\%}$$

T_{max} is from the relay datasheet (typically 85–105°C).

5.3 Contact Resistance Sub-Index

$$HI_{\text{resistance}} = 1 - \frac{R_{\text{contact}}}{R_{\text{max}}} \quad \in [0,1] \qquad \textbf{Weight: 25\%}$$

$R_{\text{max}} = 200\,\text{m}\Omega$ (critical threshold). This sub-index is the most sensitive early-warning signal – it responds to contact pitting before HI_damage or HI_cycles reach alarming levels.

5.4 Cycle Sub-Index

$$HI_{\text{cycles}} = 1 - \frac{N_{\text{cycles}}}{N_{\text{rated}}} \quad \in [0,1] \qquad \textbf{Weight: 10\%}$$

N_{rated} from datasheet (e.g. 100,000 operations).

5.5 Composite Health Index

$$\boxed{HI_{\text{final}} = \Big(0.40 \cdot HI_{\text{damage}} + 0.25 \cdot HI_{\text{thermal}} + 0.25 \cdot HI_{\text{resistance}} + 0.10 \cdot HI_{\text{cycles}}\Big) \times 100\%} \tag{1}$$

5.6 Anomaly Overrides

Certain sensor events **bypass HI thresholds** and force an immediate alarm escalation:

Override trigger	Condition	Forced action
Contact resistance spike	$R > 3\times$ baseline	Force alarm = CRITICAL
Bounce duration increase	Bounce time $> 1.5\times$ spec	Force alarm = WARNING
Thermal runaway	$dT/dt > 2\,^{\circ}\text{C/s}$	Force alarm = CRITICAL

Table 7: Anomaly overrides – bypass HI thresholds and escalate alarm directly.

6. Stage 4 – Alarm State Machine (Day 2)

6.1 Four Alarm States

State	HI	LED	Buzzer	Action Required
Healthy	$> 70\%$	Solid Green	Silent	Log data, normal operation
Warning	40–70%	Solid Yellow	1 beep/60 s	Alert dashboard + SMS to technician
Critical	20–40%	Slow blink Red	3 beeps/30 s	Maintenance work order + email
Failure Imminent	$< 20\%$	Fast blink Red	Continuous	Auto-switch to backup relay

Table 8: Alarm state machine – four states with outputs and required actions.

Key Design Decision: **Automatic Backup Switching**: when HI drops below 20%, the ESP32 asserts a GPIO to trigger a changeover relay *before* failure occurs. Wire a spare GPIO to the backup relay coil through a transistor driver. This is the difference between predictive maintenance and reactive repair.

Verification Gate (must pass before next stage): OLED shows live HI. LED changes colour at correct thresholds. Buzzer fires on schedule. NVS restores state after reboot.

7. Stage 5 – MODBUS Register Map (Day 3)

The ESP32 acts as a **MODBUS RTU slave** at 9600 baud, 8N1, over RS485 via MAX485. Use the ArduinoModbus library.

Note: Never change register addresses after the Pi polling script is deployed. Downstream code depends on them.

Addr.	Name	Type	Scale	Description
40001	N_cycles_Hi	UINT16	×1	Cycle count – high 16 bits
40002	N_cycles_Lo	UINT16	×1	Cycle count – low 16 bits
40003	HI_final	UINT16	÷1000	Composite HI (0–1000)
40004	HI_damage	UINT16	÷1000	Damage sub-index
40005	HI_thermal	UINT16	÷1000	Thermal sub-index
40006	HI_resist	UINT16	÷1000	Resistance sub-index
40007	alarm_state	UINT16	×1	0=Healthy, 1=Warning, 2=Critical, 3=Imminent
40008	T_coil	INT16	÷10	Temperature in 0.1°C units
40009	R_contact	UINT16	÷100	Contact resistance in 0.01 mΩ
40010	I_last_sw	UINT16	÷100	Current at last switch (0.01 A)
40011	V_last_sw	UINT16	÷10	Voltage at last switch (0.1 V)
40012	W_total	UINT16	×1	Accumulated arc energy (normalised)
40013	dT_dt	INT16	÷10	Temperature rate (0.1°C/s)
40014	fault_bits	UINT16	bitmap	Bit0 = R spike, Bit1 = bounce, Bit2 = thermal runaway

Table 9: MODBUS holding register map – all 4xxxx Holding Registers.

7.1 RS485 Bus Rules

- One RS485 bus supports up to 32 ESP32 slave nodes (one per relay).
- Each ESP32 gets a unique MODBUS slave address (1–32) stored in NVS.
- Use a 120 Ω termination resistor at both ends of the bus for runs >3 m.
- Maximum bus length: 1200 m at 9600 baud.
- The Pi Master polls slaves sequentially: slave 1, slave 2, ..., slave N, repeat.

Verification Gate (must pass before next stage): Use QModMaster on a laptop (USB-to-RS485 dongle). All 14 registers from every node must be readable and sane. Values must match the OLED display.

8. Stage 6 – Raspberry Pi Gateway Software (Day 4)

The Pi is the bridge between edge nodes and the cloud. Build and verify each module independently before integrating the next.

8.1 Pi Software Stack

Module	Library	Role
poll_relay.py	pymodbus	MODBUS master – polls all 14 registers from each slave
sqlite_store.py	sqlite3	Local historian – appends every poll to time-series DB
mqtt_publish.py	paho-mqtt + TLS	Publishes JSON health payloads to broker
ml_inference.py	xgboost, joblib	Loads trained model; predicts ΔHI per reading
fastapi_server.py	FastAPI, uvicorn	REST API + WebSocket for the dashboard
mosquitto	system service	MQTT broker on localhost:1883

Table 10: Raspberry Pi software modules.

8.2 Step 6.1 – MODBUS Polling Script

Build this first. Verify all registers are readable before writing to SQLite.

Listing 2: MODBUS poll script (core loop).

```
from pymodbus.client import ModbusSerialClient

client = ModbusSerialClient(port="/dev/ttyUSB0", baudrate=9600)
client.connect()

while True:
    for slave_id in range(1, N_SLAVES + 1):
        regs = client.read_holding_registers(40001, 14, slave_id)
        if not regs.isError():
            process_registers(regs.registers, slave_id)
    time.sleep(5)
```

8.3 Step 6.2 – SQLite Storage

- Always `INSERT` (append) – never overwrite. ML needs full time-series history.
- Index on `(device_id, ts)` for fast range queries.
- Store both raw register values AND computed HI.
- Checkpoint WAL every 60 s: `conn.execute('PRAGMA wal_checkpoint')`.

8.4 Step 6.3 – MQTT Publishing

Publish to topic `inv/{device_id}/rul` with QoS 1 and `retain=True`.

Listing 3: MQTT JSON payload format (inv/{id}/rul).

```
{
  "device_id":    "relay_K1",
  "ts":           1718000000,
  "hi_final":     0.742,
  "hi_damage":    0.810,
  "hi_thermal":   0.920,
  "hi_resistance": 0.680,
  "hi_cycles":    0.851,
  "alarm_state":  0,
  "t_coil":       42.5,
  "r_contact":    8.3,
  "i_last_sw":    12.4,
  "n_cycles":     14823,
  "delta_hi_pred": -0.18,
  "hi_future_pred": 73.9,
  "rul_cycles":   49000
}
```

8.5 Step 6.4 – FastAPI Backend

- `GET /api/fleet` – current health of all devices (from in-memory dict).
- `GET /api/device/{id}/history?limit=200` – SQLite query.
- `WebSocket /ws` – push every MQTT message to all connected browsers.
- `POST /api/alerts/{id}/acknowledge` – mark alert handled in DB.

Verification Gate (must pass before next stage): SQLite grows every 5 s. mosquitto_sub on Pi shows JSON arriving. /api/fleet returns JSON from browser. WebSocket push works (test with wscat).

9. Stage 7 – MQTT Topic Architecture (Day 5)

Note: Choose topic names carefully – they are permanent. Changing them later breaks all cloud subscribers.

Topic	Publisher	Payload	QoS	Retain
`inv/{id}/rul`	Pi gateway	Full health JSON	1	Yes
`inv/{id}/alarm`	Pi gateway	Alarm level + fault bitmap	1	Yes
`inv/{id}/raw`	Pi gateway	Raw sensor telemetry	0	No
`inv/{id}/event`	Pi gateway	Switching events (I, V, ts)	1	No
`fleet/summary`	Cloud	Aggregated fleet health	1	Yes
`edge/{id}/config`	Cloud	OTA threshold update	1	No

Table 11: MQTT topic structure.

10. Stage 8 – ML Pipeline: Delta-HI XGBoost (Days 6–7)

10.1 The Problem with the Old Model (Absolute HI Target)

Warning: **v1/v2 failure mode (do not repeat):** The original model predicted absolute future HI (`target = HI_future`). Training data covered HI 34–99% (healthy relay). Test data covered HI 5–44% (degraded relay). The model had never seen low-HI outputs during training, so it defaulted to predicting the training mean (≈40%) for every input – a flat, useless line.

This is the classic **covariate shift** problem.

10.2 The Fix: Predict Delta-HI

Key Design Decision: **Core change in v3:** predict the *change* in HI, not the absolute future value.

$$\Delta HI = HI[t + \texttt{FUTURE\_STEPS}] - HI[t]$$

ΔHI is always small (-3 to $+3$ percentage points) regardless of whether the relay is healthy or degraded. There is **no covariate shift** – the output distribution is the same throughout the lifecycle.

Reconstruct absolute future HI trivially at inference:

$$\widehat{HI}_{\text{future}} = HI_{\text{current}} + \widehat{\Delta HI}$$

Property	Old: absolute HI target	New: ΔHI target (v3)
Output range	0–100% (varies with relay state)	-3 to $+3$% (nearly constant)
Covariate shift	Severe	None
Generalisation	Poor on unseen HI ranges	Strong across full lifecycle
Reconstruction	Direct output	$HI_{\text{now}} + \Delta HI$

Table 12: Old vs. new prediction target comparison.

10.3 Training Data: Physics-Based Multi-Stage Degradation

Run `generate_sample_data.py` to produce `relay_data.xlsx` (3000 rows):

Phase	Rows	Degradation	What it models
Burn-in	0–300	0.00–0.12	Quick initial settling
Normal	300–2250	0.12–0.52	Slow, steady linear wear
Wear-out	2250–3000	0.52–1.00	Accelerating contact erosion

Table 13: Three-phase bathtub degradation curve for training data.

The generator also injects **25 overload events** (current ×1.8–3.5) and **12 hot spells** (+20°C for 30 readings) to ensure the model learns realistic stress events. Because all phases

and events appear throughout the dataset, an 80/20 chronological split always includes both healthy and degraded examples in both halves.

10.4 Feature Engineering

Feature Group	Features
Base sensors	current, voltage, temp, cycles, resistance, peak_current, frequency
Derived sensor	dT_dt (finite difference of temperature)
Physics outputs	HI, HI_damage, HI_thermal, HI_resistance, HI_cycles
Cumulative damage	damage (W_{total}, thermally weighted cumulative arc energy)
Lifecycle	lifecycle_pos $= t/t_{\max}$ (where in life the relay is)
HI lags	HI_lag_k for $k \in \{1, 2, 3, 5, 10, 20\}$
HI slopes	Least-squares slope of HI over windows $w \in \{5, 10, 20, 50\}$
HI rolling stats	Rolling mean & std of HI over $w \in \{5, 10, 20, 50\}$
Sensor rolling	Rolling mean & std of current, temp, resistance, voltage over $w \in \{5, 10, 20, 50\}$
Interactions	damage × temp; current × resistance; HI × cycles; HI slope × damage

Table 14: Feature engineering – 60–80 columns total.

10.5 XGBoost Hyperparameters

Parameter	Value	Rationale
n_estimators	2000	Large pool; early stopping finds optimal
max_depth	4	Shallow trees – reduces overfitting on noisy ΔHI
learning_rate	0.02	Slow, stable convergence
subsample	0.8	Row sampling – adds stochasticity
colsample_bytree	0.8	Feature sampling per tree
min_child_weight	10	Smoother predictions
gamma	0.2	Minimum split gain
reg_alpha / reg_lambda	0.1 / 2.0	L1 + L2 regularisation
early_stopping_rounds	50	Stops when val-RMSE does not improve

Table 15: XGBoost hyperparameters for ΔHI regression.

Note: `RobustScaler` (scikit-learn) is used instead of `MinMaxScaler` because it is based on median and IQR rather than min/max, making it resistant to the overload current spikes present in the data.

10.6 Train/Test Split

80% / 20% chronological split (no shuffle). The first 80% of time-ordered rows train the model; the last 20% are the held-out test set. Because we predict ΔHI rather than absolute HI, the test distribution matches training regardless of HI level.

10.7 Training Pipeline (Google Colab)

Preferred training environment: Google Colab (colab.research.google.com). The script `RelayGuard_Colab.py` is already organised as Colab cells.

1. Open Colab. Create a new notebook.
2. Paste each cell from `RelayGuard_Colab.py` into a separate code cell.
3. Run Cell 3: upload `relay_data.xlsx` (generated by `generate_sample_data.py`) or set `USE_SAMPLE_DATA = True`.
4. Run cells top to bottom. Training completes in ≈2–5 minutes.
5. Cell 8 downloads: `relay_guard_model.pkl`, `scaler.pkl`, `predictions.xlsx`, `training_results.png`.

10.8 Core Training Code

Listing 4: Delta-HI target creation and XGBoost training (from RelayGuard_Colab.py).

```
# -- Delta HI target (the key fix) ------------------------
df["HI_future"] = df["HI"].shift(-FUTURE_STEPS)
df["delta_HI"]  = df["HI_future"] - df["HI"]
df.dropna(subset=["delta_HI"], inplace=True)

# -- Features / target split ------------------------------
EXCLUDE   = {"HI_future", "delta_HI", "Alert", "time"}
feat_cols = [c for c in df.columns if c not in EXCLUDE]
X, y      = df[feat_cols].values, df["delta_HI"].values

# -- Outlier-resistant scaling ----------------------------
scaler   = RobustScaler()
X_scaled = scaler.fit_transform(X)

# -- Time-ordered 80/20 split -----------------------------
split       = int(len(X) * 0.80)
X_tr, X_te  = X_scaled[:split], X_scaled[split:]
y_tr, y_te  = y[:split], y[split:]
hi_te       = df["HI"].values[split:]   # current HI

# -- Train ------------------------------------------------
model = XGBRegressor(**XGB_PARAMS)
model.fit(X_tr, y_tr,
          eval_set=[(X_tr, y_tr), (X_te, y_te)],
          verbose=100)

# -- Reconstruct absolute HI from delta -------------------
delta_pred = model.predict(X_te)
hi_pred    = (hi_te + delta_pred).clip(0, 100)
```

10.9 Evaluation Targets

Metric	Formula	Target
MAE	$\frac{1}{n}\sum \lvert HI_{\text{actual}} - HI_{\text{pred}} \rvert$	$< 3\%$
RMSE	$\sqrt{\frac{1}{n}\sum (HI_{\text{actual}} - HI_{\text{pred}})^2}$	$< 5\%$
R^2	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	> 0.90

Table 16: ML evaluation targets (evaluated on reconstructed absolute HI, not ΔHI).

10.10 LSTM Anomaly Detection (Advanced)

Beyond XGBoost, the full system design includes an LSTM autoencoder for anomaly detection:

- **Input**: sliding window of 50 timesteps × 10 features (HI sub-indices, T, R, I, dT/dt, resistance slope, switching frequency).
- **Architecture**: Encoder LSTM(32) → bottleneck → Decoder LSTM(32) → TimeDistributed Dense(10).
- **Target**: reconstruct the input sequence (autoencoder – no failure labels needed).
- **Anomaly**: reconstruction error > mean $+3\sigma$ on healthy training set.
- **Value**: fires 10–20 data points *before* any rule-based threshold triggers. This is the AI early-warning advantage.
- Train on Google Colab GPU. Export to TFLite. Deploy `.tflite` to Pi for inference.

10.11 Weibull Survival Analysis (Probabilistic RUL)

Weibull gives a probability distribution over remaining life rather than a point estimate:

$$\text{RUL}_{50\%} = \eta \cdot (\ln 2)^{1/\beta} \tag{2}$$

$$\text{RUL}_{85\%} = \eta \cdot (-\ln 0.15)^{1/\beta} \tag{3}$$

Parameters: $\beta \approx 2.5$–3.5 (wear-out failure, increasing hazard), η fitted from B10 life data or actual field failures.

Verification Gate (must pass before next stage): `ml_inference.py` loads `relay_guard_model.pkl` and runs inference without errors. Predicted ΔHI values are in the range −3 to +3. Anomaly flag appears in API output.

10.12 Deploying the Model on the Pi

Listing 5: Inference on Raspberry Pi using saved model.

```
import joblib, numpy as np

model  = joblib.load("relay_guard_model.pkl")
scaler = joblib.load("scaler.pkl")

# Build feature vector using same physics + feature
# engineering pipeline as training, then:
```

```
X_new      = scaler.transform(df_new[feat_cols].values)
delta_hi   = model.predict(X_new)[0]

hi_current = df_new["HI"].iloc[-1]
hi_future  = float(np.clip(hi_current + delta_hi, 0, 100))

print(f"Current HI: {hi_current:.1f}%")
print(f"Predicted  : {hi_future:.1f}%")
print(f"Alert      : {classify_alert(hi_future)}")
```

11. Stage 9 – Dashboard Development (Day 6)

Build the dashboard **after** the data pipeline is live. Real sensor data makes the demo compelling; mocked data makes it unconvincing.

11.1 Dashboard Panels (Priority Order)

Priority	Panel	Contents
P1	Fleet Overview Grid	One card per relay: ID, large HI% coloured green/amber/red, alarm state, RUL estimate.
P2	HI Timeline	Line chart (last 7 days) of HI_final + sub-index lines. Vertical markers at alarm changes. Red dots = LSTM anomaly fires.
P3	Contact Resistance Trend	R_contact in mΩ over time. Horizontal lines at 50 mΩ (warning) and 200 mΩ (critical).
P4	Current vs Damage Scatter	Switching events: I_switch on x-axis, arc energy on y-axis, coloured by temperature.
P5	Work Order Panel	Auto-populates when alarm_state ≥ 2. Shows relay ID, HI, predicted failure date, acknowledge button.
Bonus	Weibull Survival Curve	Probability of survival vs remaining cycles. 50th, 85th, 95th percentile bands.

Table 17: Dashboard panels in build priority order.

11.2 Technology Options

Option	Stack	Setup time	Best for
React + Recharts	React, Recharts, WebSocket client	2–3 days	Custom UI, demo polish
Grafana + InfluxDB	Grafana OSS, InfluxDB v2, Telegraf	2–4 hours	Quick dashboard, no-code
Node-RED Dashboard	Node-RED + node-red-dashboard	1–2 hours	Fastest prototype

Table 18: Dashboard technology options.

Verification Gate (must pass before next stage): HI gauge updates in browser without page refresh. Alarm state change visible within 10 s. All 5 panels populated with live data.

12. Stage 10 – Integration and Demo (Day 7)

12.1 Full Integration Test Sequence

Run the full end-to-end pipeline manually:

1. Power on ESP32 nodes. Confirm OLED shows live HI.
2. Run Pi poll script. Confirm SQLite grows every 5 s.
3. Confirm MQTT JSON arrives on mosquitto_sub.
4. Open dashboard in browser. Confirm all panels update live.
5. Simulate stress: apply overload current to relay. Confirm HI drops and alarm escalates on dashboard.
6. Confirm work order card auto-populates at alarm_state 2.
7. Confirm backup relay GPIO fires when HI $< 20\%$.

12.2 Demo Script (3 Minutes)

Step	What you do	What the audience sees
1	Start with all relays healthy	Dashboard all green. ESP32 LED = steady green.
2	Increase stress (potentiometer or injected overload)	HI drops live on dashboard.
3	Reach Warning (≈60%)	Yellow LED. LSTM anomaly dot appears on timeline *before* rule threshold fires. Emphasise this gap.
4	Reach Critical (≈30%)	Red LED + slow beep. Work order card pops up: "Replace Relay K1 – failure in 2.1 hours".
5	Reach Failure Imminent (<20%)	Fast blink + continuous buzzer. Dashboard red. Backup relay switches automatically.
6	Business case closing	"Unplanned downtime costs X/hour. This system predicted failure 48 h early, enabling a planned replacement."

Table 19: Demo scenario script.

Note: **Always record a backup video on Day 5.** If WiFi fails or hardware misbehaves on demo day, play the video without breaking stride. Judges care about the concept and build quality – a backup demonstrates professionalism.

13. Known Issues and Design Decisions

Issue	Resolution	Status
Flat ≈40% prediction	Switched to ΔHI target (eliminates covariate shift)	Fixed in v3
MinMaxScaler distorted by spikes	Replaced with RobustScaler (IQR-based)	Fixed in v3
Training data: healthy phase only	Multi-stage bathtub curve covers all phases in every split	Fixed (data gen v2)
train_model.py uses absolute HI	Needs updating to match Colab v3 ΔHI logic	Planned
RUL estimate is linear	Adequate short-term; learned RUL head would improve long-term RUL	Future work
No real relay data yet	Physics + bathtub simulation used as surrogate	Ongoing
LCD / MOSFET / switch models	Defined in RUL doc but not yet implemented in firmware	Future work

Table 20: Known issues, resolutions, and status.

14. Production Roadmap

14.1 Phase 1 – Edge Instrumentation (Months 1–3)

- Instrument firmware with cycle counters and stress-weighted wear models for all 5 component types.
- Validate degradation models against accelerated life test data.
- Implement non-volatile health register storage.
- Expose all data over MODBUS at agreed register map.

14.2 Phase 2 – Connectivity (Months 3–5)

- Deploy MODBUS polling gateway + MQTT pipeline.
- Stand up InfluxDB / TimescaleDB time-series database.
- Build minimal fleet dashboard (RUL gauge per unit).

14.3 Phase 3 – Cloud ML (Months 5–9)

- Collect 6+ months of fleet telemetry.
- Train LSTM autoencoder and refine XGBoost ΔHI model on real data.
- Deploy Weibull survival models fitted to actual field failures.
- Enable OTA threshold updates from cloud back to edge devices.

14.4 Phase 4 – Integration and Scale (Months 9–11)

- Connect work-order triggers to CMMS (SAP PM, IBM Maximo, or webhooks).
- Build fleet-wide RUL heatmap.
- Add confidence intervals to RUL estimates.
- Establish automated monthly retraining pipeline with new field data.

15. Team Responsibility Matrix

Role	Stages	Day	Deliverable
Hardware Lead	0, 1, 2, 3	Day 3	All sensors verified, MODBUS readable from laptop
Backend Lead	4, 5, 6	Day 5	Pi pipeline live, API serving JSON, WebSocket push working
ML Lead	8	Day 6	XGBoost deployed on Pi, inference returns delta HI
Frontend Lead	9	Day 6	Dashboard live with real data, all 5 panels visible
Full Team	10	Day 7	End-to-end test, demo rehearsal, backup video recorded

Table 21: Team responsibility matrix.

16. Summary – Complete Project Checklist

RelayGuard – Complete Build Checklist

1. **Day 0:** Tools installed. Pin map agreed. Git repo created.
2. **Day 1:** Each sensor (ACS712, ZMPT101B, MLX90614, INA219, relay GPIO) independently verified.
3. **Day 2:** FreeRTOS firmware running. OLED shows live HI. LEDs respond. NVS survives reboot.
4. **Day 3:** MODBUS slave on ESP32. QModMaster reads all 14 registers from Pi.
5. **Day 4:** Pi gateway polling. SQLite growing. MQTT JSON on broker.
6. **Day 5:** FastAPI backend live. WebSocket push working. Full backup video recorded.
7. **Day 6:** React/Grafana dashboard live with all 5 panels. XGBoost ΔHI model deployed on Pi.
8. **Day 7:** End-to-end system test. Stress scenario drives full alarm journey (green to red). Work order auto-creates. Demo rehearsed.
9. **ML:** Target MAE $< 3\%$, RMSE $< 5\%$, $R^2 > 0.90$ on reconstructed absolute HI.
10. **Architecture:** Predict ΔHI (not absolute HI) to eliminate covariate shift.